



University
of Glasgow

W9-02: Cassandra Gossiping and Load Balancing

COMPSCI4064 (H) and COMPSCI5088 (M)

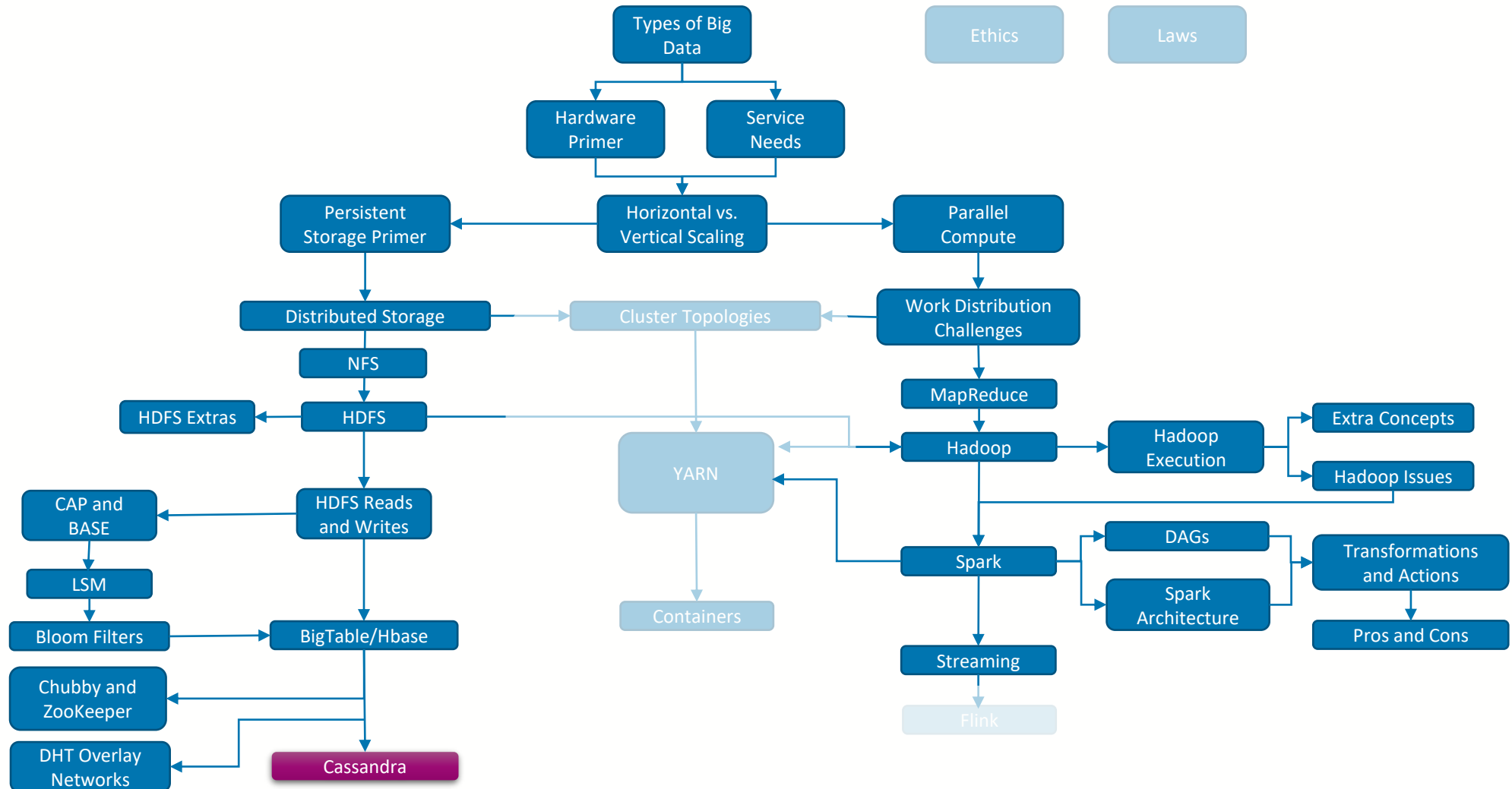
Dr. Richard McCreadie

WORLD
CHANGING
GLASGOW

A WORLD
TOP 100
UNIVERSITY



Where are we?





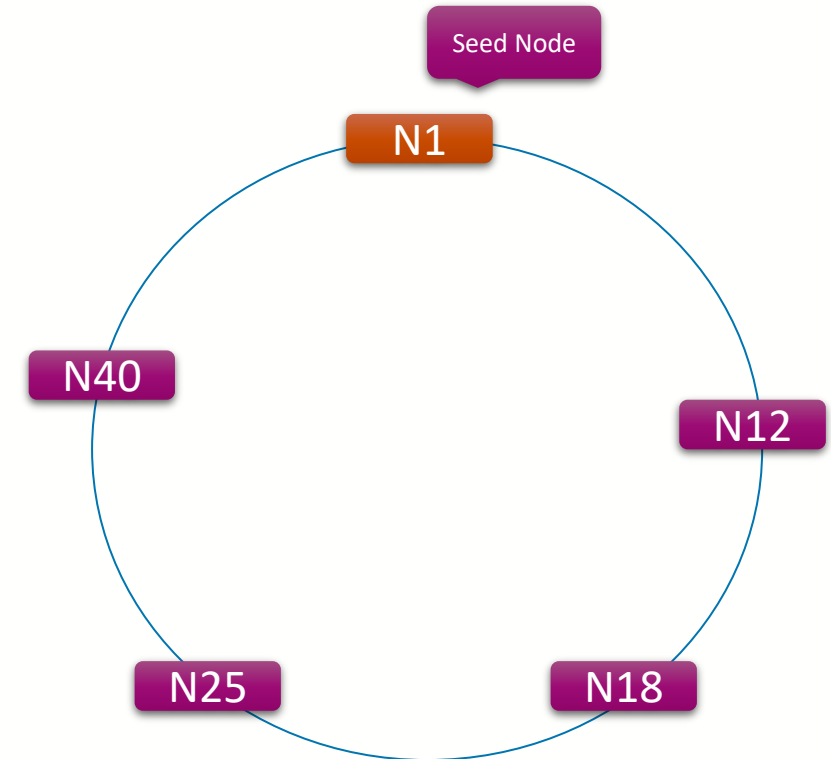
How to find data?

- This is a bit different to a normal distributed hash table in that there are **no finger tables**
 - “Classic” DHTs were designed for **highly dynamic P2P networks**
 - Need to be able to route from any node to any node without central global knowledge and minimize the cost of maintaining a consistent routing state
 - So, these systems limit the per-node routing state to $O(\log N)$ entries in N -node networks
 - Cassandra designed for operation in **Data Centres**
 - Can accrue global knowledge about the existence and location of all nodes
 - BUT do so in a decentralized way!
 - No master node storing such knowledge
 - Uses a **gossiping protocol**
 - Knowledge floods/propagates through the network
 - Eventually all nodes become aware of it
 - A.k.a. epidemic propagation / epidemic protocols



Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

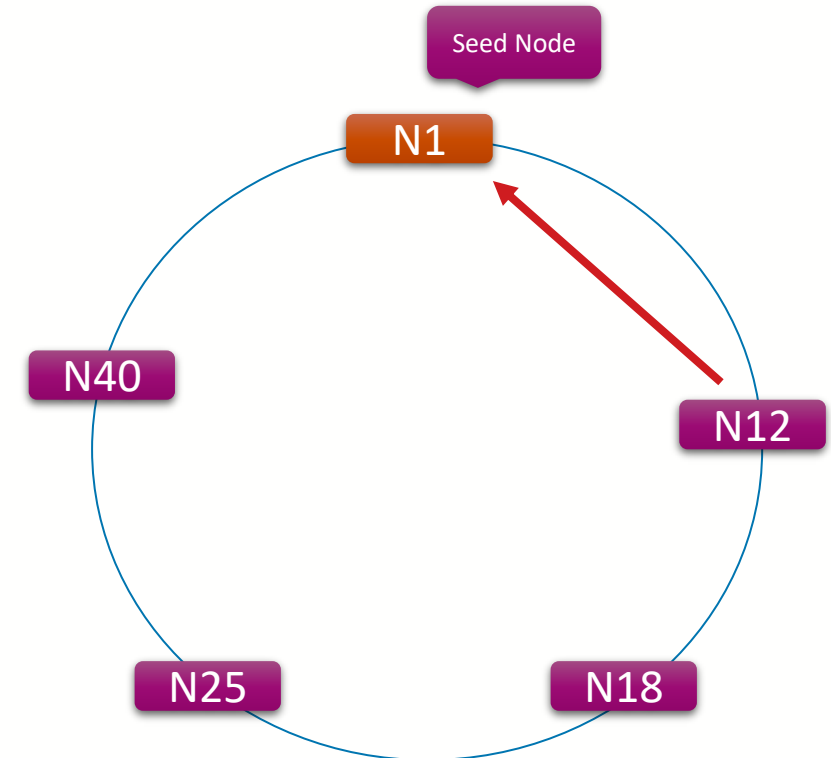




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2

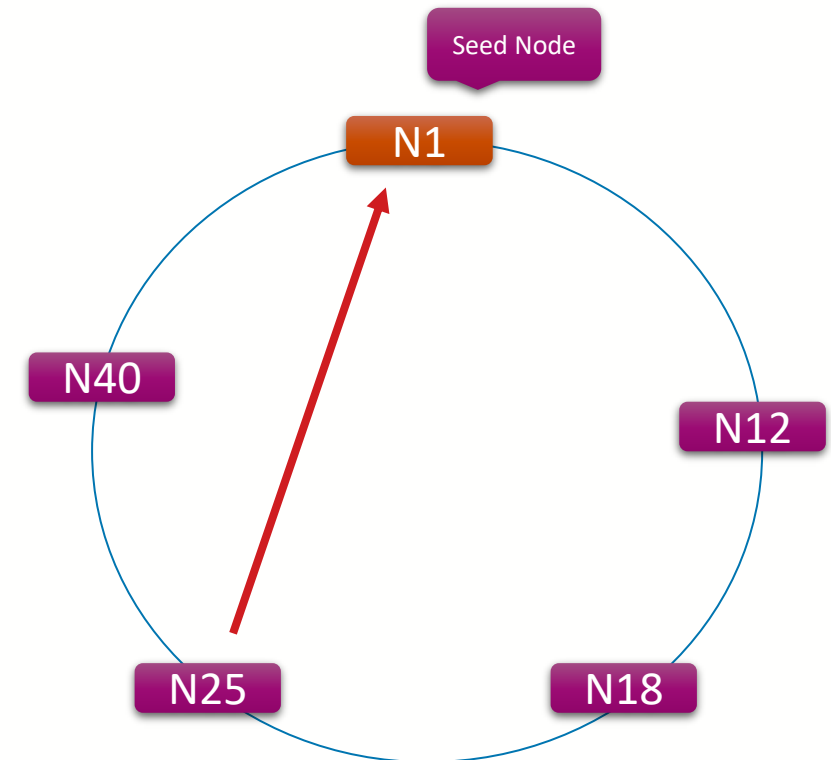




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2

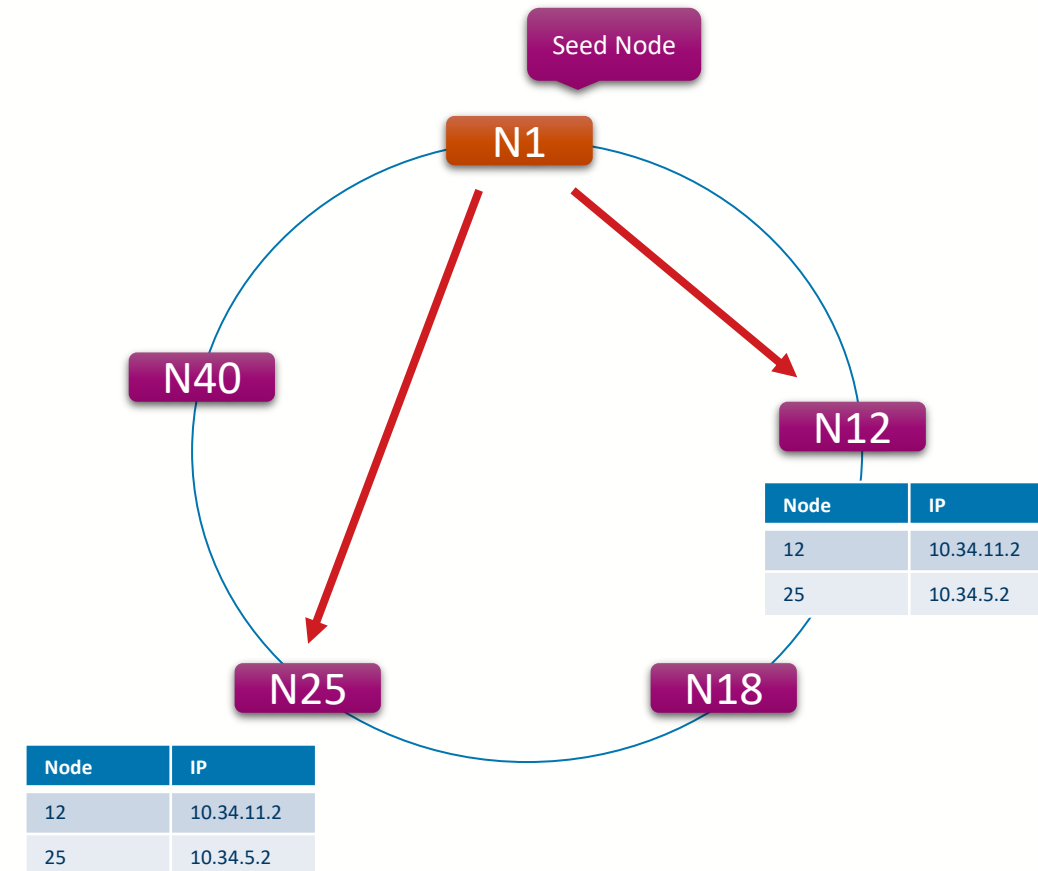




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2

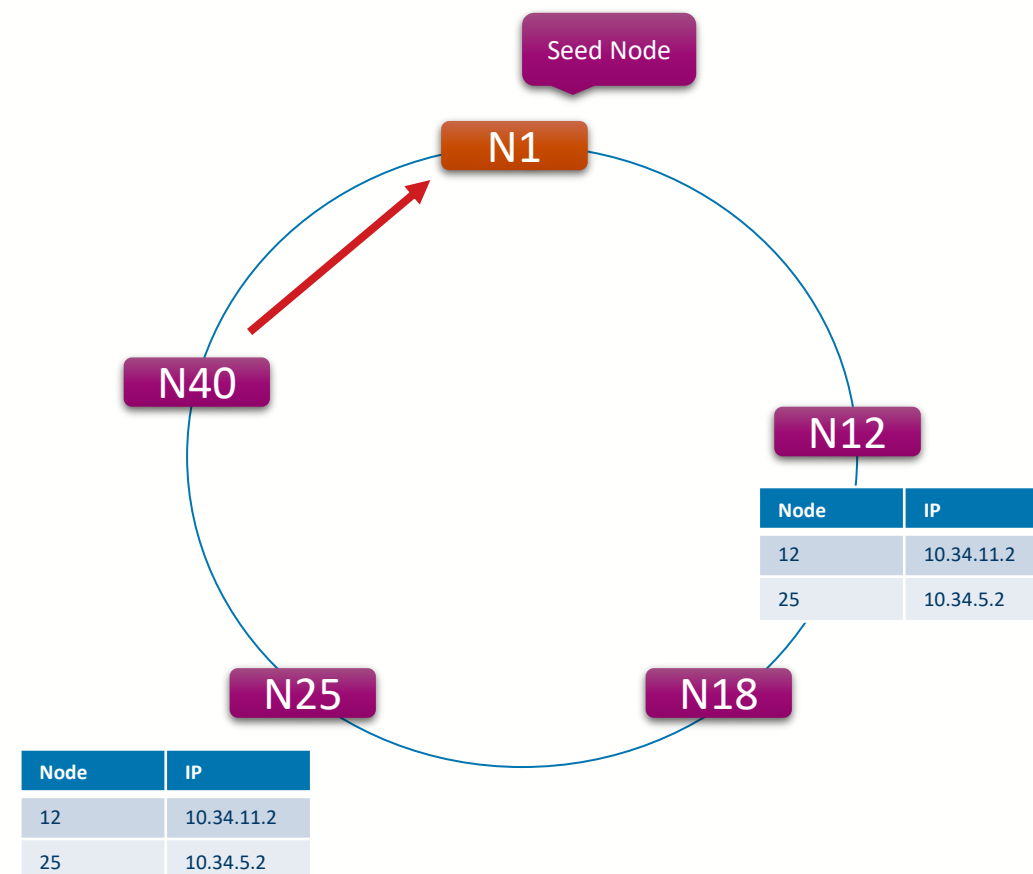




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2
40	10.43.5.30

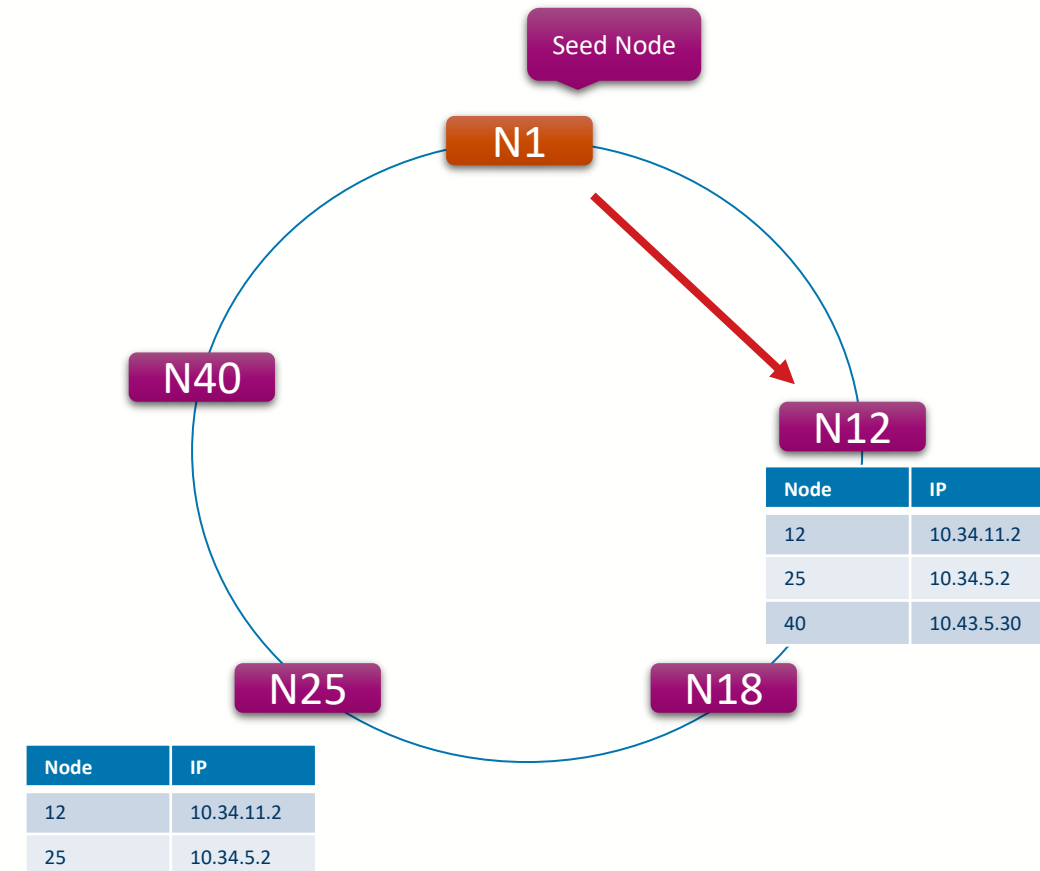




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2
40	10.43.5.30



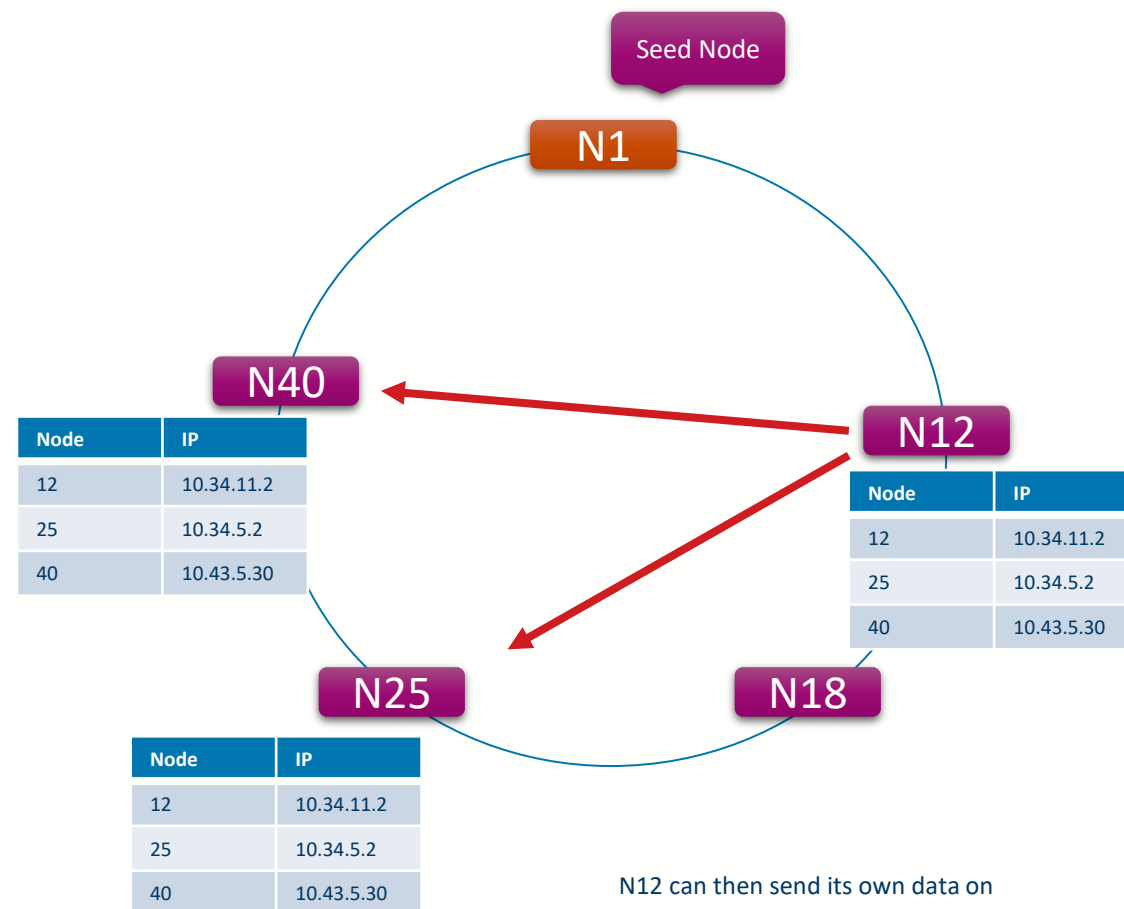
For illustration lets assume N1 only sends one message (rather than 3)



Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2
40	10.43.5.30

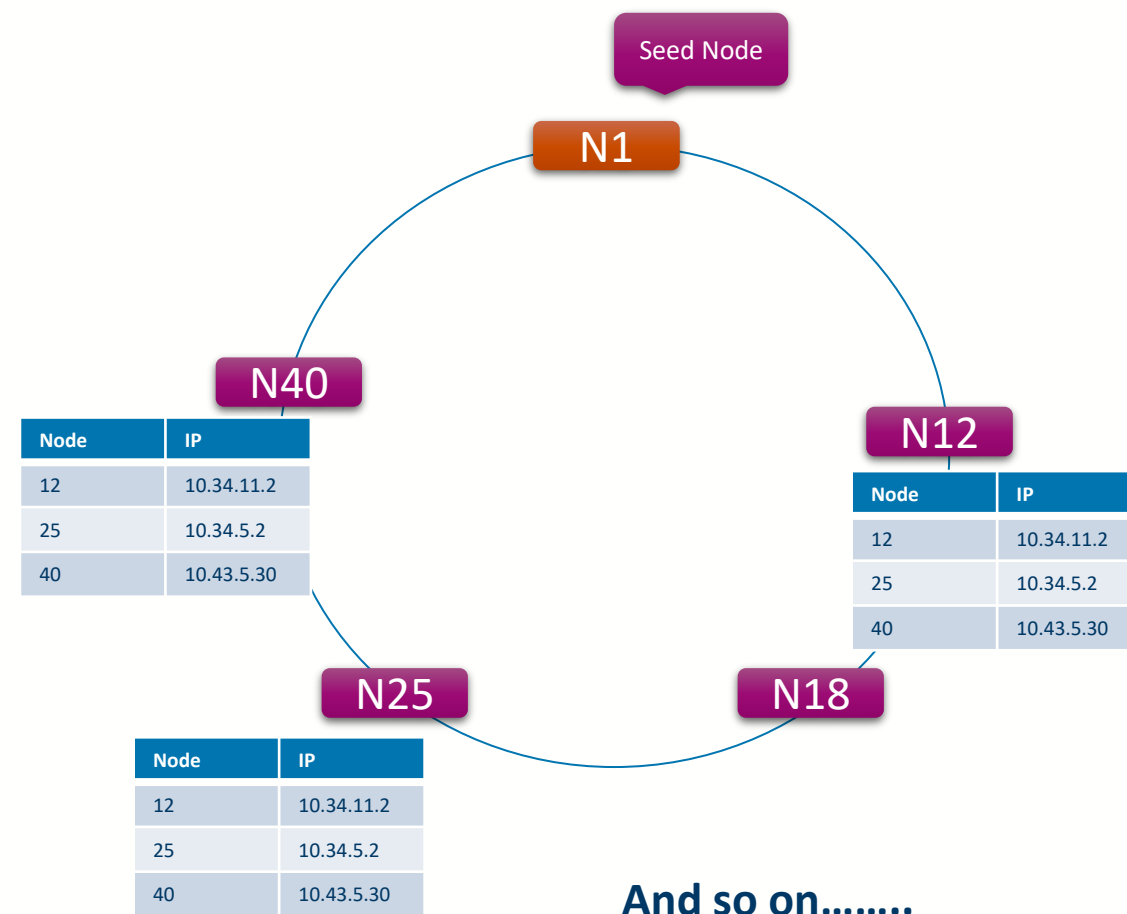




Cassandra Gossiping

- Nodes regularly (every second) generate a **token**
 - This contains information about themselves, e.g. **ID** and **Ring Position**
 - ... as well as **information about other nodes** they have communicated with
- Once a token is generated, it is **sent to 3 other nodes**
 - Eventually, each node knows of
 - All other nodes in the system
 - i.e. global knowledge
 - AND the arc of the ring they are responsible for
 - (recall all rows for a partition key will exist within an arc)
 - New nodes read a seed file to get a list of seed nodes
- Since we can assume **eventual global knowledge**, if a data request arrives at a node it just needs to **lookup its list of nodes** (which should be complete)

Node	IP
12	10.34.11.2
25	10.34.5.2
40	10.43.5.30





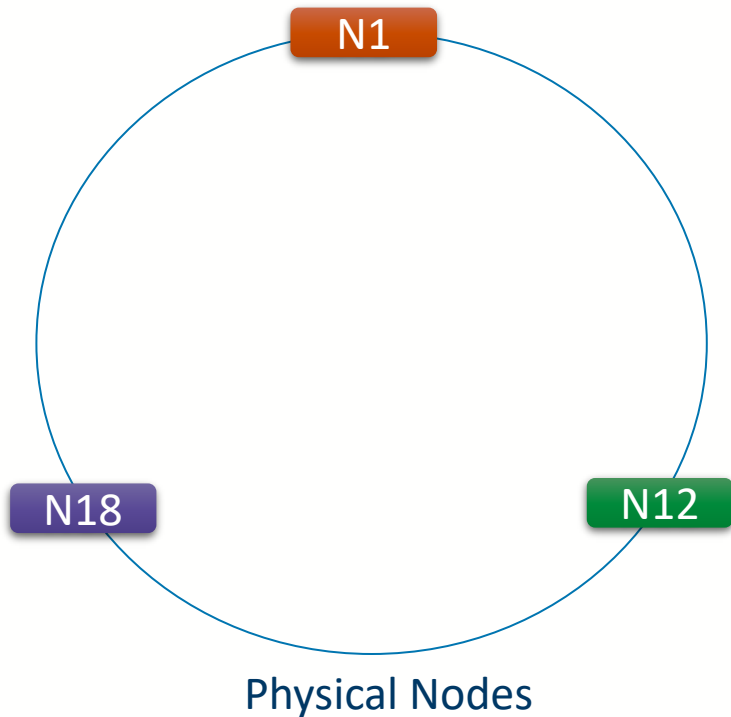
Virtual Nodes

- In this setup we might run into load balancing issues
 - Data is allocated to nodes based on a hash function
 - Might be random, or order preserving
 - But this does not necessarily spread data around
 - Some nodes might have more capacity than others
 - E.g. faster CPU or storage
- Solution:
 - Represent each node as multiple virtual nodes instead
 - Spread-out the virtual nodes for a physical node around the ring



Virtual Nodes

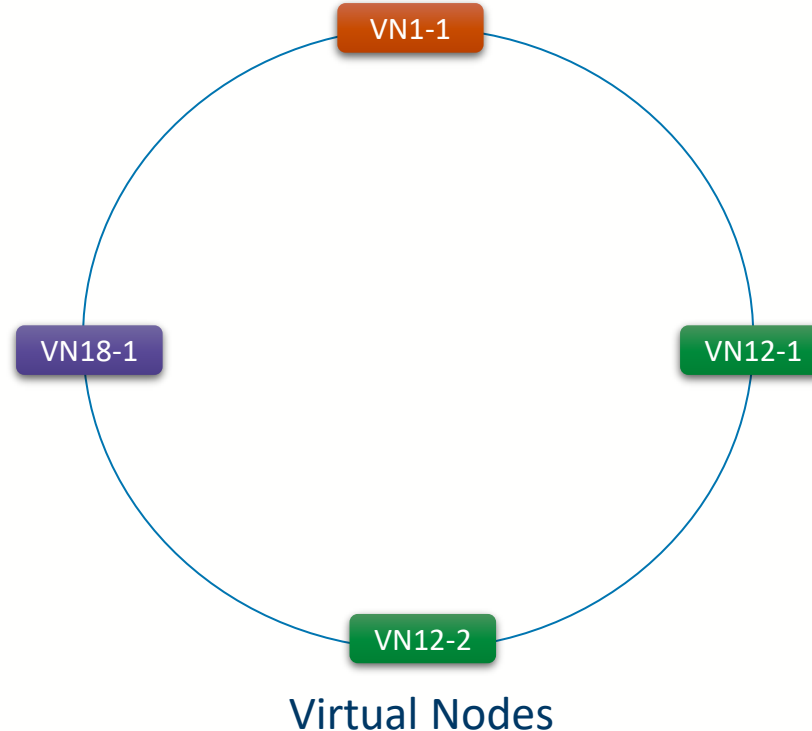
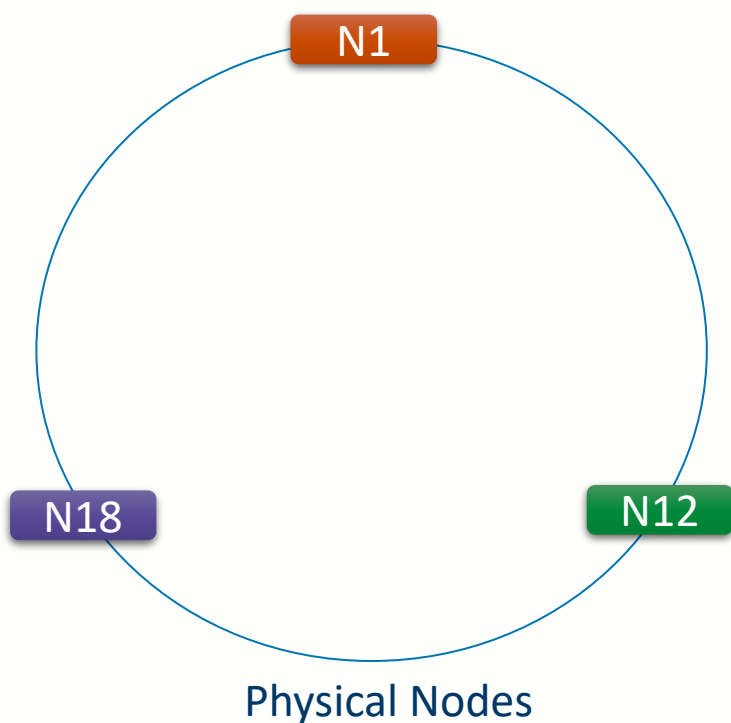
- Lets assume we have 3 machines (physical nodes)
 - N1: 4 CPU cores
 - N12: 8 CPU cores
 - N18: 4 CPU cores





Virtual Nodes

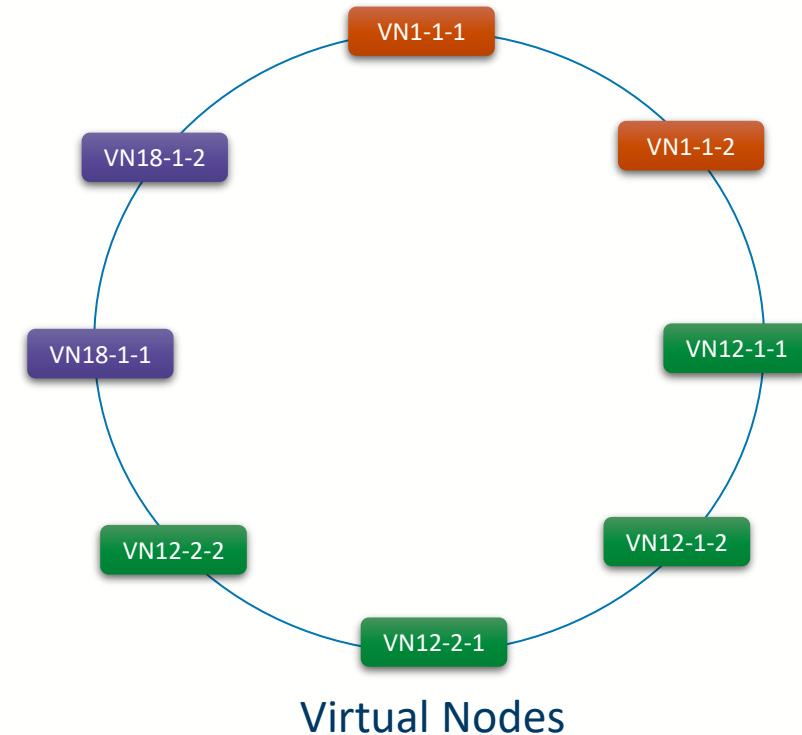
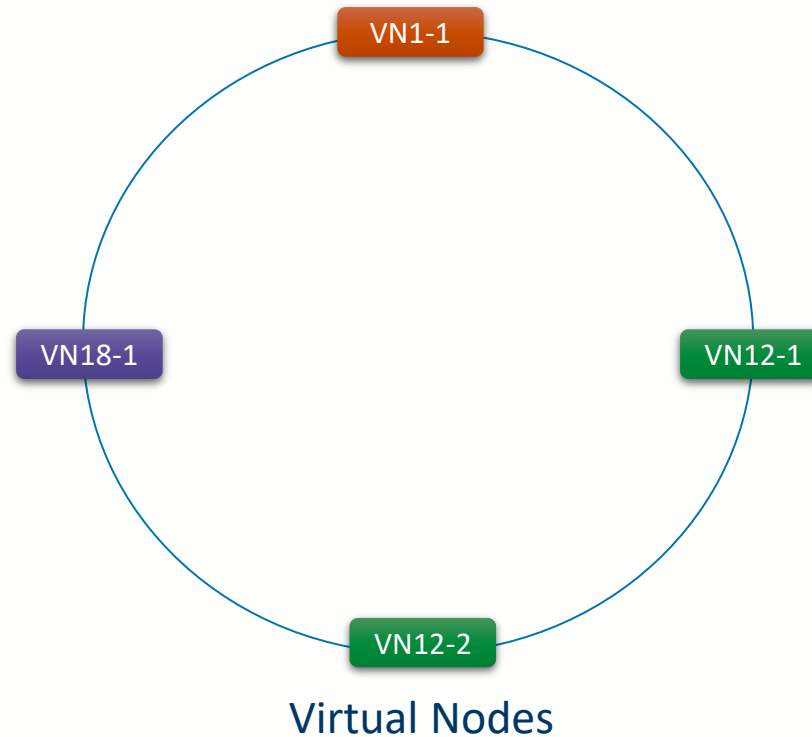
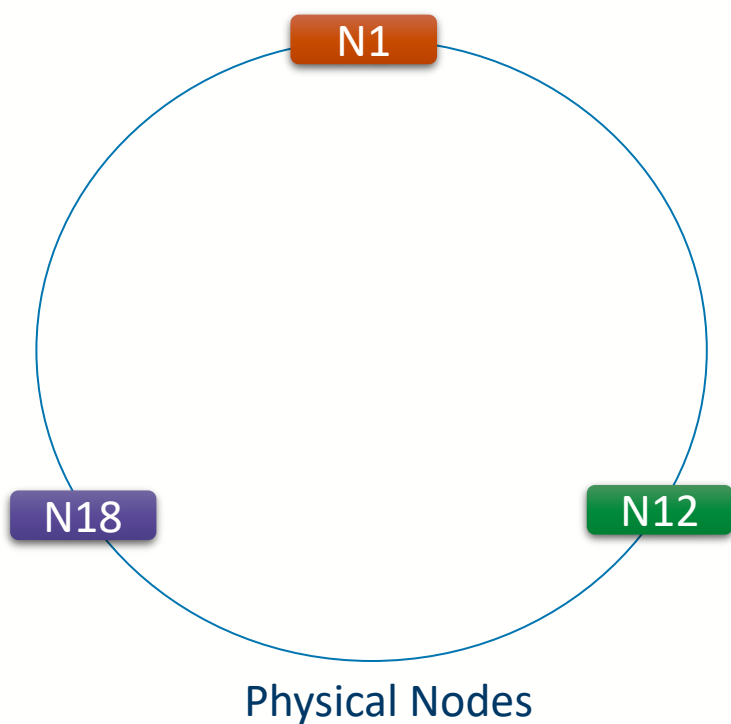
- Lets assume we have 3 machines (physical nodes)
 - N1: 4 CPU cores
 - N12: 8 CPU cores
 - N18: 4 CPU cores
- We might map these physical nodes to virtual ones based on capacity
 - N1 -> VN1-1
 - N12 -> VN12-1, VN12-2
 - N18 -> VN18-1





Virtual Nodes

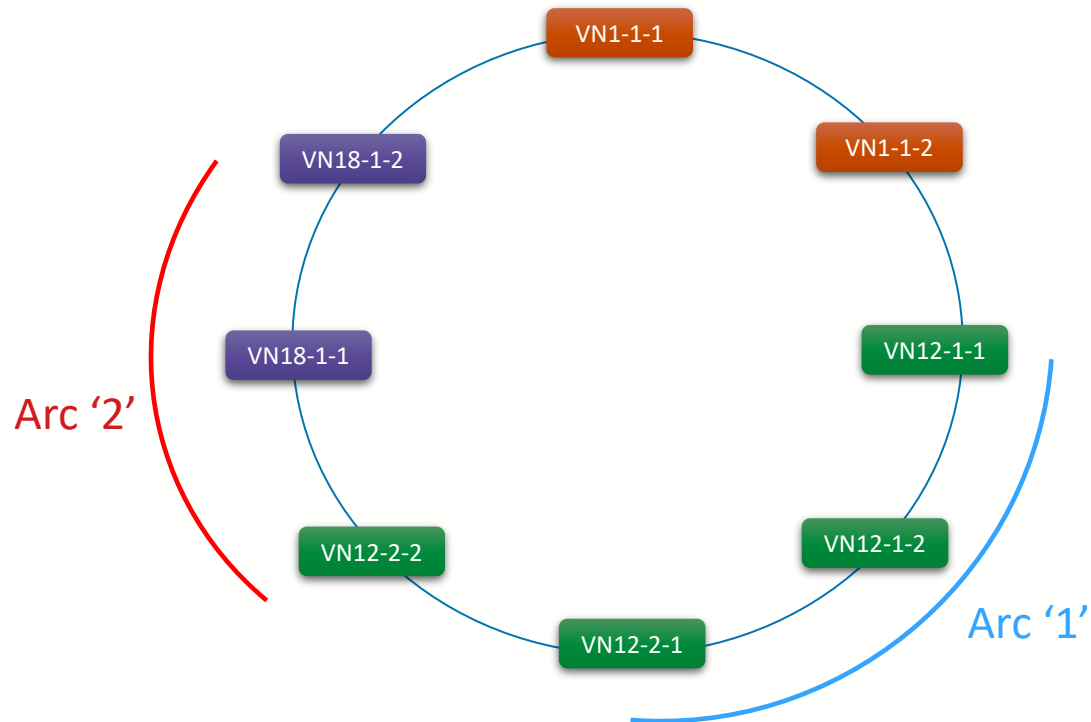
- Lets assume we have 3 machines (physical nodes)
 - N1: 4 CPU cores
 - N12: 8 CPU cores
 - N18: 4 CPU cores
- We might map these physical nodes to virtual ones based on capacity
 - N1 -> VN1-1
 - N12 -> VN12-1, VN12-2
 - N18 -> VN18-1
- Then further split the nodes...
 - VN1-1 -> VN1-1-1, VN1-1-2
 - VN12-1 -> VN12-1-1, VN12-1-2
 - VN12-2 -> VN12-2-1, VN12-2-2
 - VN18-1 -> VN18-1-1, VN18-1-2





Virtual Nodes

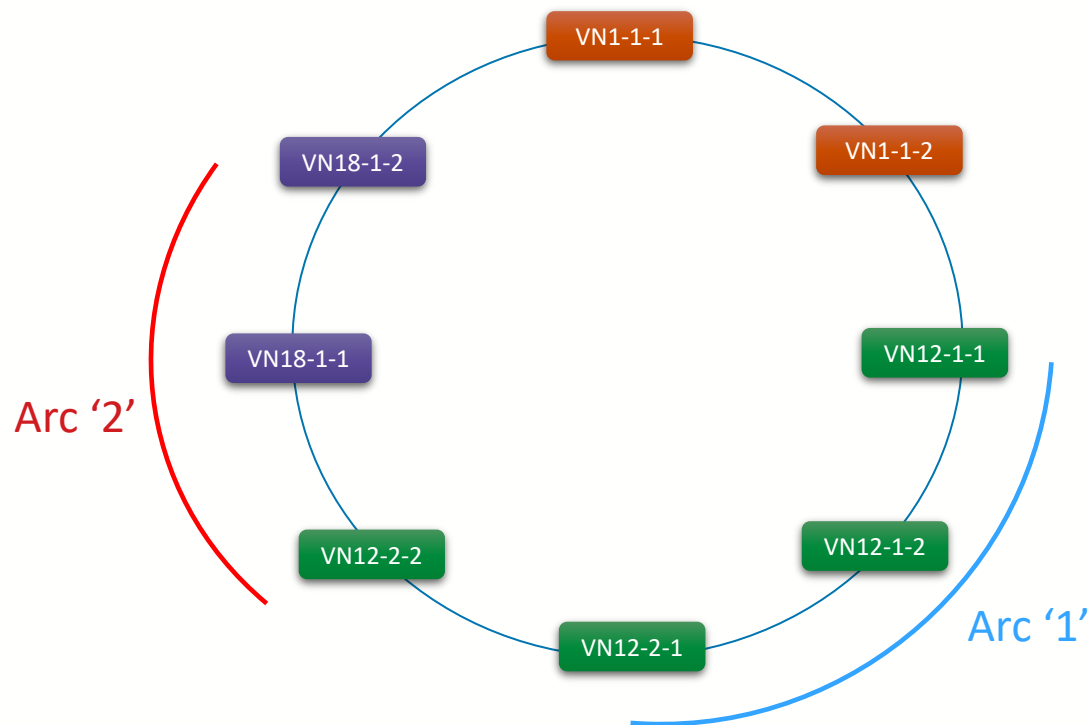
- The issue with this setup is that we have not solved the load balancing issue, as few nodes will manage each arc



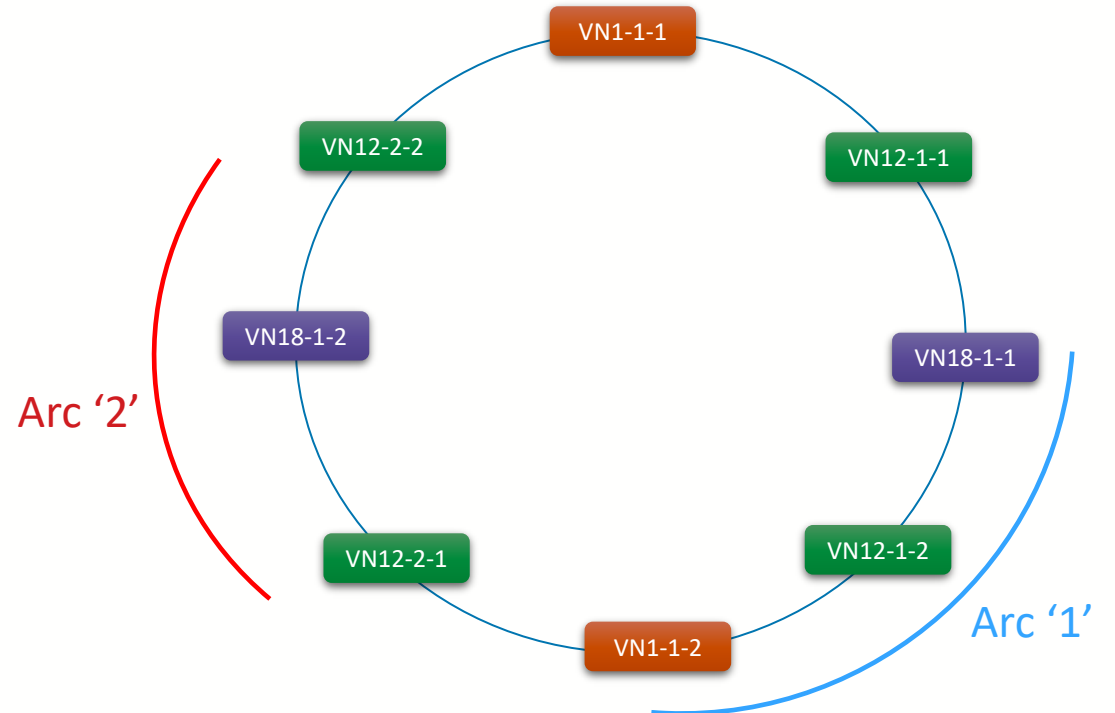


Virtual Nodes

- The issue with this setup is that we have not solved the load balancing issue, as few nodes will manage each arc



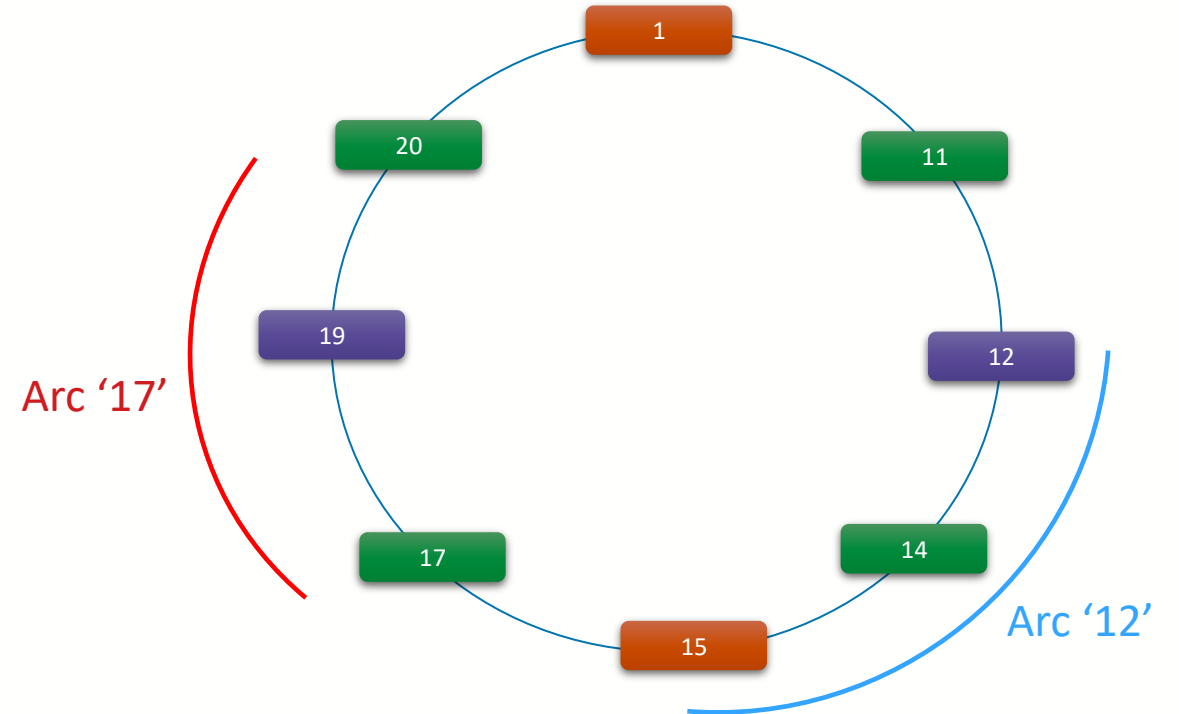
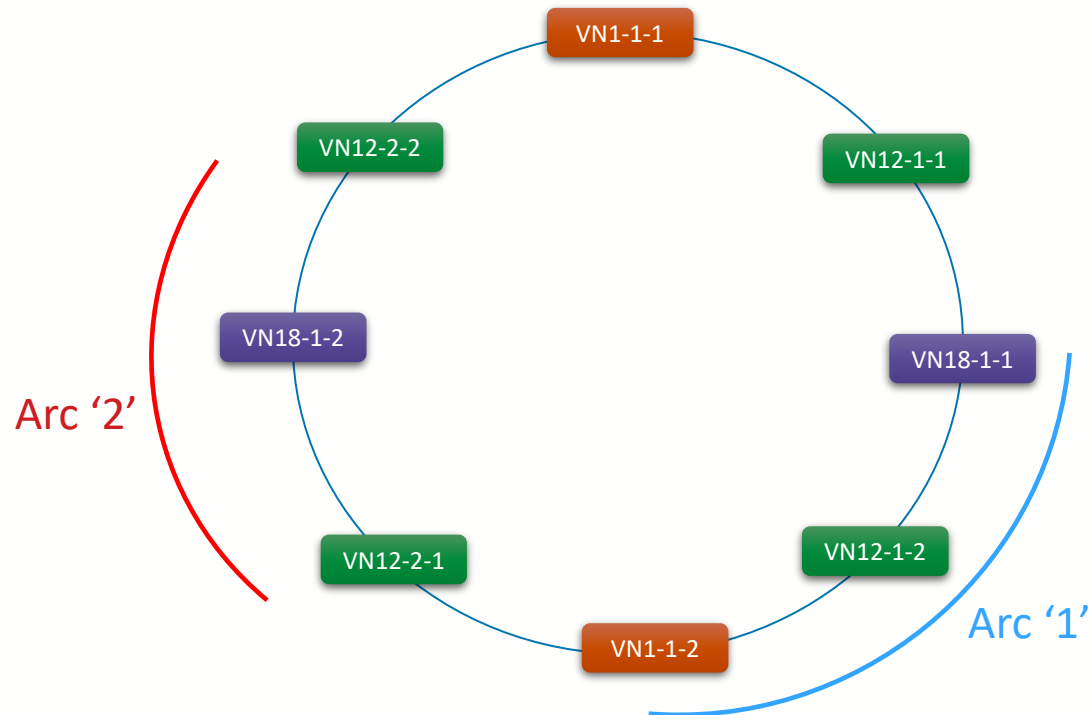
- So instead, let's spread out the virtual nodes for each physical node around the ring





Virtual Nodes

- For clarity, remember that virtual nodes still have normal numbers, just like physical nodes did





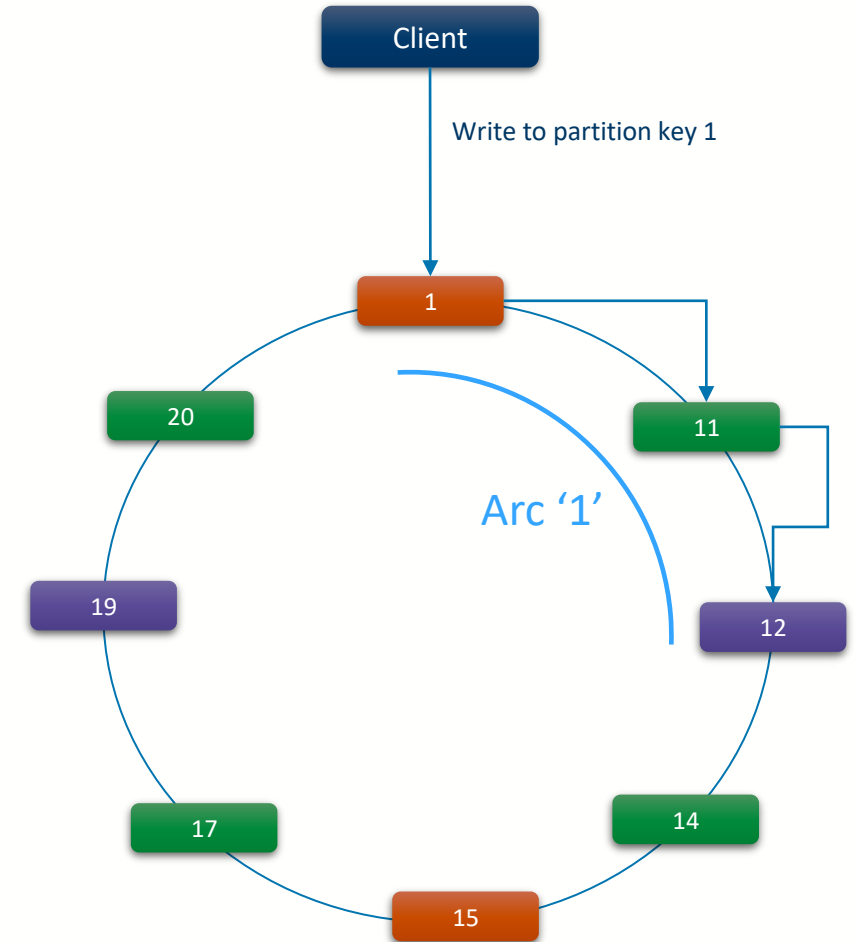
Thinking about Consistency...

- I said earlier that Cassandra focuses on **Partition Tolerance** and **Availability**
 - What is the problem with consistency?
- If there are several replicas...
 - How can I know if a replica is **up-to-date or stale**?
 - Can I read from **any** replica?
 - Do I need to read from **several**?
 - Is this issue related to **how many replicas are updated during writes**?
- Such issues are dealt with using a **replication control protocol**
 - Let's consider some options...



Option 1: Eventual Consistency

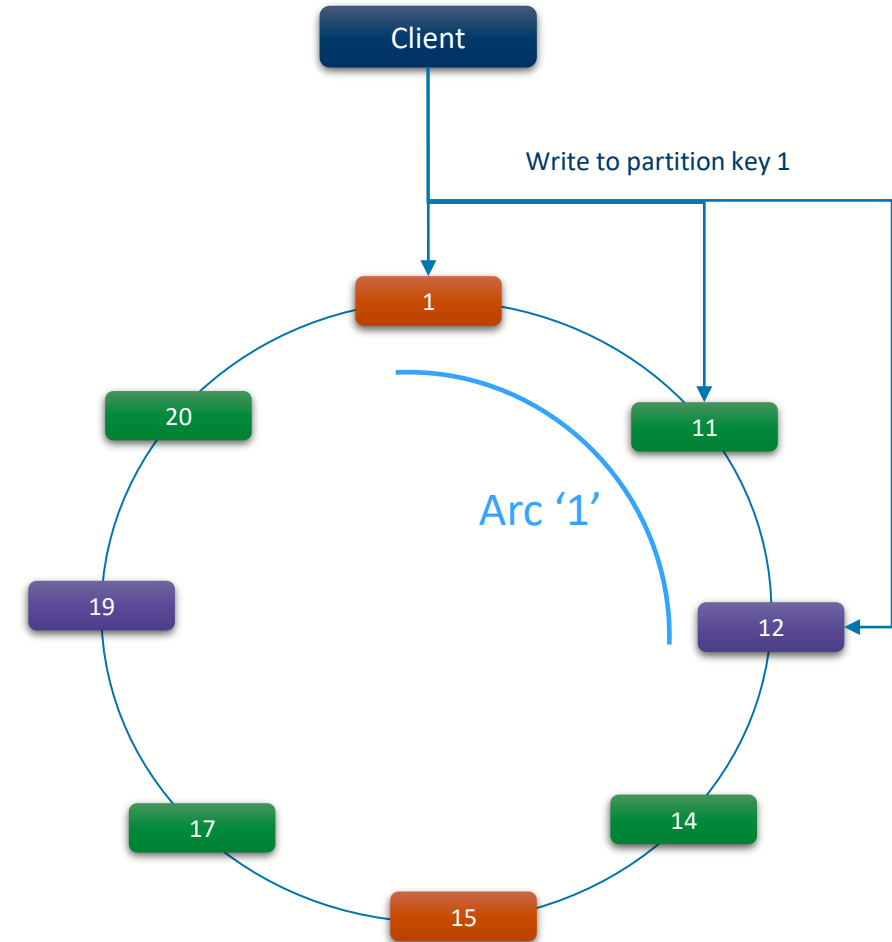
1. Client
 1. Write(1, DATA)
 2. Node 1
 1. Commit Write(1, DATA)
 2. Acknowledge Write(1, DATA)
 3. Write(11, DATA)
 3. Node 11
 1. Commit Write(11, DATA)
 2. Acknowledge Write(11, DATA)
 3. Write(12, DATA)
 4. Node 12
 1. Commit Write(12, DATA)
 2. Acknowledge Write(12, DATA)
- Assuming no failures, we will reach the target replication
 - Any replica that has finished commit can serve that data
 - BUT it is **not guaranteed to be the most recent copy**
 - Data propagation could be taking place and not reached that node yet – there is no way to know





Option 2: Majority Consensus

1. Client
 1. Write(1, DATA)
 2. Write(11, DATA)
 3. Write(12, DATA)
 4. Wait for majority of acknowledgements
 2. Node 1
 1. Acknowledge Write(1, DATA)
 3. Node 11
 1. Acknowledge Write(11, DATA)
 4. Node 12
 1. Acknowledge Write(12, DATA)
 5. Client
 1. Commit Write(1, DATA)
 2. Commit Write(11, DATA)
 3. Commit Write(12, DATA)
- Only commits a write if the **majority of nodes have the data**
 - Implies a **two phase protocol**
 - Phase 1: ensure that at least a majority are available; if not, abort write operation
 - Phase 2: send writes to at least a majority
 - This solution is **guaranteed to provide the most recent data** upon a read





Consistency Vs. Performance

- Enforced consistency in Option 2 has serious performance implications!
- For **Reading**:
 - Reading the replica closest to me incurs the smallest delay
 - So, if I can READ FROM ANY, it'd be great!
 - Waiting for just one replica node to return the data is much faster than waiting for several!
 - So, if I can READ FROM ONE, it'd be great!
- For **Writing**:
 - Writing to one (or fewer) replicas is much faster than writing to many!
- However, having a primary replica has the disadvantages of:
 - Risk for performance bottlenecks
 - Reduced consistency



Option 3: Quorum

- A generalization of majority consensus
 - Assume that instead of a majority, we can **contact less nodes** during reads or writes
 - **Replicas N** : The number of replicas
 - **Read Quorum R** : The number of nodes that must be contacted during a read
 - **Write Quorum W** : The number of nodes that must be contacted during a write
- **Quorum** works if **both**:
 - $R + W > N$
 - $2 * W > N$
- **Advantages**
 - Guaranteed intersection of all operations at a single replica
 - All READs guaranteed to see latest version!
- Tuneable trade-offs
 - For read-intensive objects / apps: r can be smaller ($\rightarrow w$ is larger)
 - For write-intensive objects / apps: w can be smaller ($\rightarrow r$ is larger)



Cassandra Customisation

Write Consistency

Level	Behaviour
ANY	Written to at least 1 node(including HH)
ONE	1 replica's commit log and memory table
QUORUM	$N/2+1$ replicas
LOCAL_QUORUM	$N/2+1$ replicas within local D.C.(only with cross D.C. strategy)
EACH_QUORUM	$N/2+1$ replicas within each D.C.(only with cross D.C. strategy)
ALL	Written to all replicas

Read Consistency

Level	Behaviour
ANY	Not supported
ONE	Returns record returned by first replica to respond
QUORUM	Returns record with most recent timestamp once at least $N/2+1$ replicas reported.
LOCAL_QUORUM	Returns record with most recent timestamp once at least $N/2+1$ replicas reported within local D.C.
EACH_QUORUM	Returns record with most recent timestamp once at least $N/2+1$ replicas reported within each D.C.
ALL	Returns record with most recent timestamp once all replicas have responded.



Cassandra Vs. HBase

Similarities

- Both instances of LSM trees
 - Plus per-file index, Bloom filters, compaction rounds, etc.
- Same (pretty much) data model
- Data partitioned horizontally
 - Although not at “region boundaries” as in BigTable/Hbase
 - Default partitioner: RandomPartitioner (MD5-hash)
- Data replicated across nodes

Differences

- Cassandra generally faster overall
- No single point of failure in Cassandra
 - (no Master → each node a possible “master”)
- Easily supports multi-datacentre deployments
- Somewhat easier to use out of the box (OpsCenter, CQL)



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow